



GT-R: Masked DINOv3 Representation Distillation for Detection Transformers with Gated Linear Attention

Anonymous authors

Paper under double-blind review

“If all you want from a car is speed, you should buy a Nissan GT-R!” — Jeremy Clarkson, The Sunday Times.

Abstract

1 Introduction

1.1 Preliminaries

Gated Linear Attention Transformer. The Gated Linear Attention Transformer (GLA) Yang et al. (2024) is an efficient sequence modeling layer that combines linear attention with a data-dependent gating mechanism. Given an input sequence $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_L]^\top \in \mathbb{R}^{L \times d}$, where L denotes the sequence length and d denotes the hidden dimension, GLA first projects each token into query, key, and value representations:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \quad (1)$$

where $\mathbf{W}_Q, \mathbf{W}_K$, and $\mathbf{W}_V$ are learnable projection matrices. For the t -th token, GLA computes a data-dependent gate to control how historical information is retained in the recurrent state:

$$\begin{aligned} \alpha_t &= \sigma(\mathbf{x}_t \mathbf{W}_\alpha + b_\alpha)^{1/\tau}, \\ \beta_t &= \sigma(\mathbf{x}_t \mathbf{W}_\beta + b_\beta)^{1/\tau}, \\ \mathbf{G}_t &= \alpha_t^\top \beta_t, \end{aligned} \quad (2)$$

where $\sigma(\cdot)$ is the sigmoid function, b_α and b_β are bias terms, and τ is a temperature parameter. The gated recurrent computation is then formulated as:

$$\begin{aligned} \mathbf{S}'_{t-1} &= \mathbf{G}_t \odot \mathbf{S}_{t-1}, \\ \mathbf{S}_t &= \mathbf{S}'_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t, \\ \mathbf{o}_t &= \mathbf{q}_t \mathbf{S}_t, \\ \mathbf{r}_t &= \text{Swish}(\mathbf{x}_t \mathbf{W}_r + \mathbf{b}_r), \\ \mathbf{y}_t &= (\mathbf{r}_t \odot \text{LN}(\mathbf{o}_t)) \mathbf{W}_O, \end{aligned} \quad (3)$$

where $\mathbf{S}_t$ is the recurrent state, $\odot$ denotes element-wise multiplication, $\text{LN}(\cdot)$ denotes layer normalization, and $\mathbf{r}_t$ is an input-dependent output gate. In the following sections, we use $\text{GLA}(\cdot)$ to denote the gated linear attention computation over an input sequence.

The framework of DETR-like models. DETR-like detectors follow an end-to-end object detection framework built upon an image backbone, a transformer encoder, a transformer decoder, and prediction heads Carion et al. (2020). Given an input image, the backbone first extracts multi-scale visual features, which are flattened into image tokens and fed into the transformer encoder for feature refinement. A set of object queries $\mathbf{Q}_{obj} = \{\mathbf{q}_1, \dots, \mathbf{q}_N\}$ is then used as object candidates in the transformer decoder. The decoder updates these object queries by modeling query-query interactions through self-attention and query-image interactions through cross-attention. Finally, the refined object queries are passed to classification and box regression heads to predict object categories and bounding boxes. This framework provides a query-based detection pipeline, where object localization and recognition are performed by decoding a fixed set of learnable object queries.

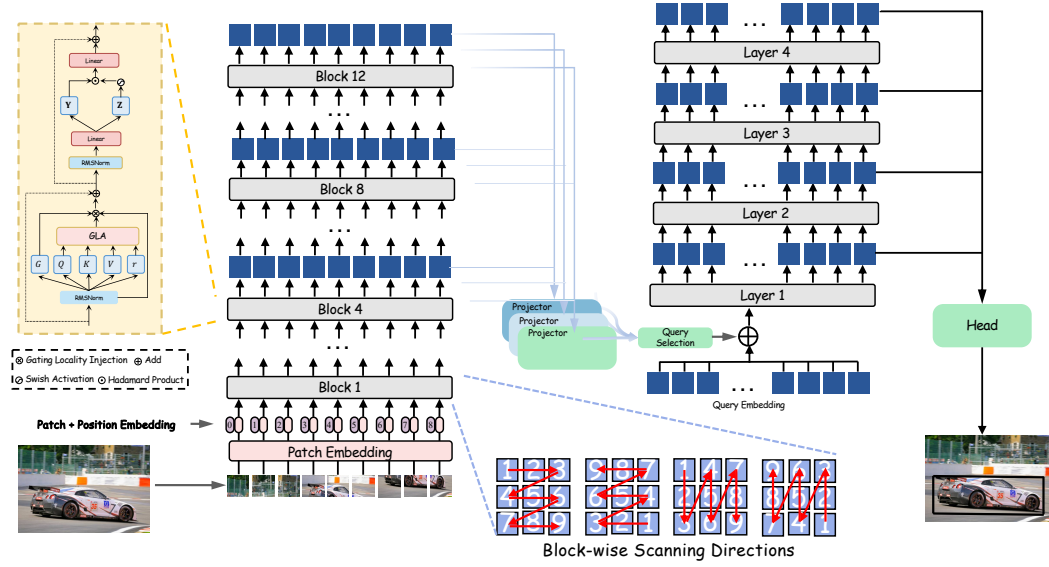


Figure 1: Overview of GTR. A 12-layer GLA backbone alternates two-dimensional scan directions and exposes intermediate features to a lightweight three-scale projector. The resulting feature pyramid is decoded by a query-based detection head.

2 Method

2.1 Overview

We propose GTR, a DETR-like detector that transfers detection-specialized DINOv3 representations into a gated linear attention (GLA) backbone. As shown in Fig. 1, GTR has three detector components: a GLA-based visual backbone, a lightweight multi-scale feature encoder, and a query-based detection decoder. The backbone replaces quadratic self-attention with recurrent gated linear attention, while the encoder and decoder keep the set-prediction interface of modern DETR-style detectors.

The main difficulty is not only to build a faster backbone, but also to make a recurrent token mixer inherit the dense representation quality of a strong self-attention teacher. We therefore first prepare a detection-specialized DINOv3 teacher following the EdgeCrafter paradigm Liu et al. (2026), and then distill this teacher into the GLA backbone with a one-stage masked distillation objective. Different from multi-stage Transformer-to-SSM distillation pipelines Bick et al. (2024; 2025); Wang et al. (2024), our distillation directly optimizes the GTR backbone with masked token prediction, intermediate Gram supervision, and a new Visible Representation Alignment (VRA) loss. The following subsections describe the teacher construction and distillation objective, the GLA backbone, the multi-scale encoder, the detection decoder, and the chunk-based GLA inference kernel.

2.2 Masked DINOv3 Representation Distillation

Detection-specialized DINOv3 teacher. A generic pretrained DINOv3 model provides strong visual representations, but its features are not explicitly optimized for dense detection localization. Before distilling GTR, we therefore construct a detection-specialized teacher by adapting DINOv3 to object detection with an ECDet-style formulation following EdgeCrafter Liu et al. (2026). This teacher is kept frozen during student distillation. The purpose of this step is to convert a general-purpose DINOv3 representation into a task-aware target that contains object-level semantics, spatial correspondence, and localization-sensitive feature geometry.

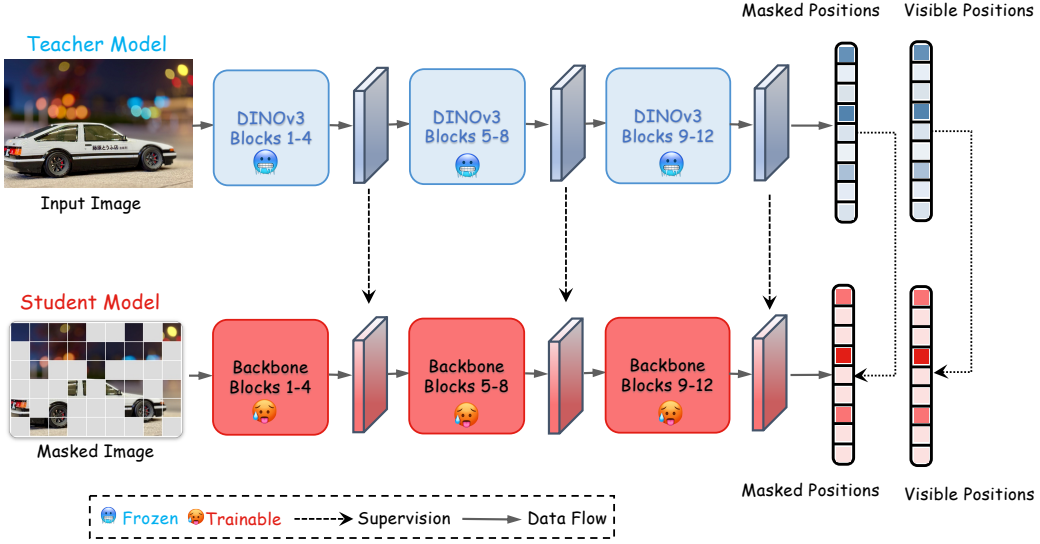


Figure 2: Masked DINOv3 representation distillation. The frozen detection-specialized teacher processes the complete image, while the trainable GTR student receives masked patches. Intermediate visible-token relations and final representations at both masked and visible positions provide the distillation targets.

Distillation setup. Figure 2 summarizes the one-stage distillation pipeline. Given an input image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$, the frozen teacher receives the complete image, while the GTR student receives a masked patch sequence. A subset of patch positions is replaced by a learnable mask token, and we denote the masked and visible token sets by Ω_m and Ω_v , respectively. The teacher and student produce final token representations $\mathbf{Y}_{\text{tea}}, \mathbf{Y}_{\text{stu}} \in \mathbb{R}^{L \times d}$, where L is the number of patch tokens and d is the teacher feature dimension. When the student hidden dimension differs from the teacher dimension, a lightweight linear projection $\phi(\cdot)$ maps the student features into the teacher feature space. We use a mask ratio of 0.5 by default.

Masked token prediction. Masked token prediction transfers the final semantic target of the detection-specialized teacher to the GLA student. Following masked image distillation methods Peng et al. (2023); Wei & Chellappa (2025), we supervise the student at masked positions with an ℓ_2 regression loss:

$$\mathcal{L}_{\text{mask}} = \frac{1}{|\Omega_m|} \sum_{i \in \Omega_m} \|\phi(\mathbf{Y}_{\text{stu}}^i) - \mathbf{Y}_{\text{tea}}^i\|_2^2. \quad (4)$$

This objective forces the student to infer the teacher representation of unseen tokens from the visible context. However, for a heterogeneous recurrent backbone, masked prediction alone does not fully constrain how visible tokens are represented during GLA-based token mixing.

Visible Representation Alignment. Visible tokens require an explicit constraint when the student replaces self-attention with GLA. Empirically, a self-attention student can keep its visible-token features close to the teacher even when only masked tokens are supervised, because the teacher and student share a similar global token-mixing mechanism. In contrast, a GLA student updates tokens through an order-dependent recurrent state, so optimizing only the masked positions can still allow the visible-token representations to drift away from the teacher. This visible representation drift is harmful for dense detection because visible tokens form the spatial feature maps consumed by the encoder and decoder.

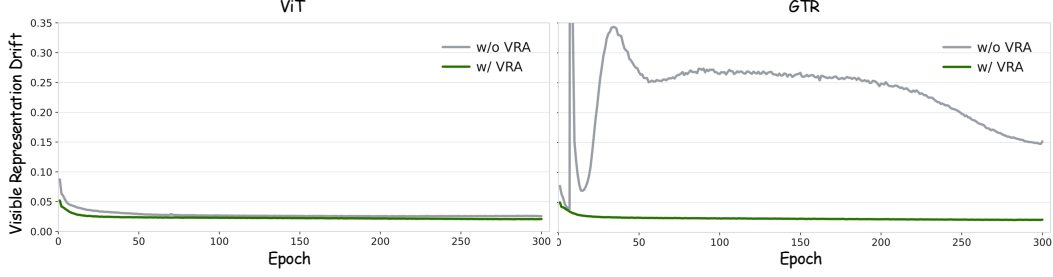


Figure 3: Visible representation drift during distillation. The drift is measured using the visible-token feature discrepancy defined by equation 5.

To address this problem, we introduce Visible Representation Alignment (VRA), which directly aligns the final-layer visible tokens:

$$\mathcal{L}_{\text{vra}} = \frac{1}{|\Omega_v|} \sum_{i \in \Omega_v} \|\phi(\mathbf{Y}_{\text{stu}}^i) - \mathbf{Y}_{\text{tea}}^i\|_2^2. \quad (5)$$

We also use the discrepancy in equation 5 as a diagnostic measure of visible representation drift during training. As shown in Fig. 3, removing VRA causes only a limited increase in visible representation drift for the ViT student, but leads to a substantial and persistent drift for the GLA student. Applying VRA consistently suppresses this discrepancy, with a markedly larger effect on the GLA student. This observation empirically supports the need for explicit visible-token alignment when transferring self-attention representations to a recurrent GLA backbone.

VRA complements masked token prediction with a seen-token constraint. Intuitively, $\mathcal{L}_{\text{mask}}$ teaches the student to predict teacher features for unseen tokens, while $\mathcal{L}_{\text{vra}}$ preserves the teacher’s representation geometry for tokens that are actually visible to the student.

Intermediate Gram supervision. Intermediate Gram supervision transfers the teacher’s token-to-token relational structure rather than only its final output representation. We apply this supervision only to the visible token features from the 4-th, 8-th, and 12-th backbone layers, i.e., $K = 3$ selected stages. Let $\mathbf{F}_{\text{tea}}^k$ and $\mathbf{F}_{\text{stu}}^k$ denote the visible token features from the k -th selected stage of the teacher and student, respectively. We restrict this supervision to visible tokens to avoid leaking masked content into the student, and compute pairwise cosine affinity matrices:

$$\begin{aligned} \mathbf{A}_{\text{tea}}^k(i, j) &= \frac{\mathbf{F}_{\text{tea}}^k(i) \cdot \mathbf{F}_{\text{tea}}^k(j)}{\|\mathbf{F}_{\text{tea}}^k(i)\|_2 \|\mathbf{F}_{\text{tea}}^k(j)\|_2}, \\ \mathbf{A}_{\text{stu}}^k(i, j) &= \frac{\mathbf{F}_{\text{stu}}^k(i) \cdot \mathbf{F}_{\text{stu}}^k(j)}{\|\mathbf{F}_{\text{stu}}^k(i)\|_2 \|\mathbf{F}_{\text{stu}}^k(j)\|_2}. \end{aligned} \quad (6)$$

The Gram loss is then defined as:

$$\mathcal{L}_{\text{gram}} = \frac{1}{K|\Omega_v|} \sum_{k=1}^K \sum_{i \in \Omega_v} \left[1 - \langle \bar{\mathbf{A}}_{\text{tea}}^k(i, :), \bar{\mathbf{A}}_{\text{stu}}^k(i, :) \rangle \right], \quad (7)$$

where $\bar{\mathbf{A}}(i, :)$ denotes the ℓ_2 -normalized i -th row of the affinity matrix. This loss encourages the student to mimic the teacher’s intermediate token relations. Following the motivation of DINOv3 Siméoni et al. (2025), such relational supervision is particularly relevant to dense prediction because local structure and fine-grained spatial correspondence must be preserved across layers. The overall distillation objective is:

$$\mathcal{L}_{\text{distill}} = \lambda_{\text{mask}} \mathcal{L}_{\text{mask}} + \lambda_{\text{vra}} \mathcal{L}_{\text{vra}} + \lambda_{\text{gram}} \mathcal{L}_{\text{gram}}, \quad (8)$$

where λ_{mask} , λ_{vra} , and λ_{gram} balance the three losses. Unless otherwise specified, all loss weights are set to 1.

2.3 GTR Backbone

Since GLA scans tokens as a one-dimensional sequence, GTR alternates the scan order across blocks to reduce directional bias on the two-dimensional image grid. Specifically, lightweight flip and transpose operations make consecutive blocks scan the patch grid from left to right, right to left, top to bottom, and bottom to top. All returned features are restored to the original spatial order, and we use the outputs of the 4-th, 8-th, and 12-th backbone layers as multi-level representations:

$$\{\mathbf{X}_4, \mathbf{X}_8, \mathbf{X}_{12}\} = \text{GTRBackbone}(\mathbf{I}), \quad (9)$$

where each $\mathbf{X}_l$ is restored to the original patch order and reshaped into a stride-16 spatial feature map.

2.4 Multi-scale Feature Encoder

The GLA backbone produces strong token features at a single stride, but object detection requires multi-scale features for objects of different sizes. We therefore construct a lightweight three-level feature pyramid from the tapped backbone layers.

Let $\mathbf{F}_4, \mathbf{F}_8, \mathbf{F}_{12} \in \mathbb{R}^{\frac{H}{16} \times \frac{W}{16} \times C}$ denote the reshaped feature maps from the 4-th, 8-th, and 12-th backbone layers. For each target stride $s \in \{8, 16, 32\}$, we first align all selected features to the same spatial resolution:

$$\tilde{\mathbf{F}}_l^{(s)} = \mathcal{R}_s(\mathbf{F}_l), \quad l \in \{4, 8, 12\}, \quad (10)$$

where $\mathcal{R}_8(\cdot)$ denotes upsampling, $\mathcal{R}_{16}(\cdot)$ denotes identity mapping, and $\mathcal{R}_{32}(\cdot)$ denotes down-sampling. For each target scale, the aligned features from different semantic depths are concatenated along the channel dimension and fused by a lightweight C2f module:

$$\mathbf{P}^{(s)} = \text{C2f}_s \left(\text{Concat}[\tilde{\mathbf{F}}_4^{(s)}, \tilde{\mathbf{F}}_8^{(s)}, \tilde{\mathbf{F}}_{12}^{(s)}] \right), \quad s \in \{8, 16, 32\}. \quad (11)$$

This produces a three-level feature pyramid $\{\mathbf{P}^{(8)}, \mathbf{P}^{(16)}, \mathbf{P}^{(32)}\}$, also denoted as $\{\mathbf{P}_3, \mathbf{P}_4, \mathbf{P}_5\}$, with unified channel dimensions. Compared with heavier feature pyramid encoders, this design keeps the encoder compact while aggregating early, middle, and late backbone representations at every detection scale. The three pyramid levels $\mathbf{P}_3$, $\mathbf{P}_4$, and $\mathbf{P}_5$ are then flattened and concatenated as the multi-scale memory for the decoder.

2.5 Query-based Detection Decoder

The decoder follows the DETR-style set prediction paradigm Peng et al. (2025). Given the multi-scale features $\{\mathbf{P}^{(8)}, \mathbf{P}^{(16)}, \mathbf{P}^{(32)}\}$, we flatten and concatenate them into memory tokens:

$$\mathbf{M} = \text{Concat}[\text{Flatten}(\mathbf{P}^{(8)}), \text{Flatten}(\mathbf{P}^{(16)}), \text{Flatten}(\mathbf{P}^{(32)})]. \quad (12)$$

Initial anchor proposals are generated from the multi-scale memory, and the Top- K candidates with the highest classification responses are selected to initialize object queries.

During training, we adopt grouped queries Chen et al. (2023) by dividing detection queries into $G = 3$ groups with shared decoder parameters. This provides denser supervision and improves query diversity, while only the primary group is used during inference.

2.6 Efficient Chunk-based GLA Inference

Direct recurrent GLA inference contains a token-level state dependency, which limits parallelism when implemented one token at a time. We reduce this dependency by materializing chunk boundary states. For each head, we use a key-wise gated GLA formulation by setting the value-side gate to an all-one vector.

Let $\mathbf{g}_t \in \mathbb{R}^{1 \times d_k}$ denote the log decay vector and $\gamma_t = \exp(\mathbf{g}_t)$. The recurrent update is: $\mathbf{S}_t = \text{Diag}(\gamma_t) \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t$, $\mathbf{o}_t = \text{scale} \cdot \mathbf{q}_t \mathbf{S}_t$. This formulation is equivalent to applying the matrix gate $\mathbf{G}_t = \gamma_t^\top \mathbf{1}_{d_v}$ to the recurrent state in Eq. 3.

Building on chunkwise GLA Yang et al. (2024), we partition the sequence into $N = \lceil L/C \rceil$ chunks with chunk size C . For chunk c and local index r , the within-chunk cumulative log decay is:

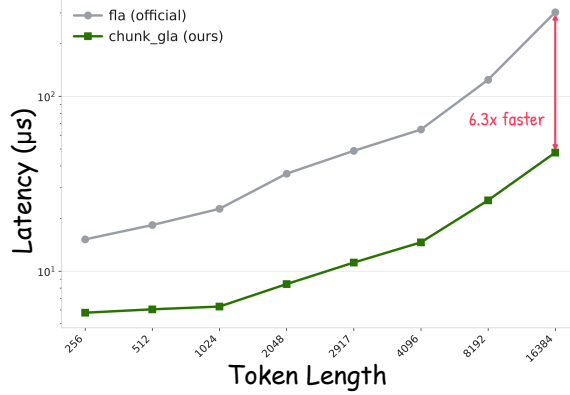


Figure 4: Latency comparison between the official FLA implementation and our chunk-based GLA implementation. The vertical axis shows inference latency on a base-2 logarithmic scale.

$$\mathbf{A}_{c,r} = \sum_{i=0}^r \mathbf{g}_{c,i}. \quad (13)$$

The contribution of all tokens inside chunk c to the next boundary state is:

$$\mathbf{U}_c = \sum_{r=0}^{C_c-1} (\mathbf{k}_{c,r} \odot \exp(\mathbf{A}_{c,C_c-1} - \mathbf{A}_{c,r}))^\top \mathbf{v}_{c,r}, \quad (14)$$

where C_c is the valid length of chunk c . Let $\mathbf{S}_{[c]}$ denote the recurrent state at the beginning of chunk c . The chunk boundary transition is:

$$\mathbf{S}_{[c+1]} = \text{Diag}(\mathbf{d}_c) \mathbf{S}_{[c]} + \mathbf{U}_c, \quad \mathbf{d}_c = \exp(\mathbf{A}_{c,C_c-1}). \quad (15)$$

The key optimization is to treat each chunk as an affine state transition and compute all boundary states with a Kogge–Stone parallel prefix scan. Each chunk is represented by a transition pair $(\mathbf{d}_c, \mathbf{U}_c)$. Given two consecutive transitions, their composition is:

$$(\mathbf{d}_b, \mathbf{U}_b) \circ (\mathbf{d}_a, \mathbf{U}_a) = (\mathbf{d}_b \odot \mathbf{d}_a, \text{Diag}(\mathbf{d}_b) \mathbf{U}_a + \mathbf{U}_b). \quad (16)$$

This operator is associative, so the boundary recurrence in Eq. 15 can be evaluated in $O(\log N)$ scan stages instead of a serial scan over N chunks.

After the boundary states are materialized, each chunk computes its outputs independently:

$$\begin{aligned} \mathbf{o}_{c,r} = & \text{scale} \cdot (\mathbf{q}_{c,r} \odot \exp(\mathbf{A}_{c,r})) \mathbf{S}_{[c]} \\ & + \text{scale} \cdot \sum_{j=0}^r \left[(\mathbf{q}_{c,r} \odot \exp(\mathbf{A}_{c,r})) (\mathbf{k}_{c,j} \odot \exp(-\mathbf{A}_{c,j}))^\top \right] \mathbf{v}_{c,j}. \end{aligned} \quad (17)$$

The first term reads information accumulated before the current chunk, and the second term computes causal interactions inside the chunk.

As shown in Fig. 4, our chunk-based implementation consistently reduces inference latency compared to the official FLA Yang & Zhang (2024) implementation, with a more pronounced advantage for longer sequences. At a token length of 16,384, our implementation achieves a $5.9\times$ speedup.

3 Experiments

3.1 Experimental Setup

Datasets and metrics. For the main detector comparison, each GTR variant is first pre-trained on Objects365 Shao et al. (2019) and then fine-tuned on COCO Lin et al. (2014) train2017. The downstream detectors used in the ablation and instance segmentation experiments are instead trained directly on COCO from the corresponding masked-distilled backbone, without Objects365 detector pre-training. We evaluate on COCO val2017 with AP averaged over IoU thresholds from 0.50 to 0.95, together with AP_{50} , AP_{75} , and AP for small, medium, and large objects. We report box AP for detection and mask AP for instance segmentation.

Training protocol. All model training is performed on eight NVIDIA A100 GPUs. For the main comparison, the S/M/L/X backbones are initialized from their corresponding masked-distilled GLA checkpoints, after which the complete detectors are pre-trained on Objects365 for 36 epochs with a global batch size of 128. For COCO fine-tuning, we transfer the Objects365-pre-trained backbone, reinitialize the feature encoder and detection decoder, and train for 30 epochs with a global batch size of 32. The Objects365 and COCO stages use AdamW Loshchilov & Hutter (2019), a flat-cosine learning-rate schedule, exponential moving average, and 640×640 evaluation inputs. The architecture-dependent learning rates, augmentations, loss weights, and model configurations are reported in Appendix A.

Efficiency evaluation. We remeasure every detection and segmentation model in a unified FP16, batch-size-one environment on a single NVIDIA GeForce RTX 4090. Tables 1 and 2 report the minimum, mean, and median per-image latency; compilation, graph capture, autotuning, and warm-up are excluded from timing. DETR-style models are timed for the network forward pass, whereas the timing for YOLO variants that require NMS also includes CUDA NMS. Because deployment conversion, input resolution, and post-processing differ across model families, the complete protocol is specified in Appendix B.

3.2 Comparison with Real-Time Detectors

Overall trade-off. GTR forms a strong accuracy-latency frontier from the small to the extra-large regime. GTR-S/M/L/X obtain 52.7/55.6/57.7/58.1 AP with minimum latencies of 1.139/1.350/1.738/1.931 ms, respectively. The mean and median measurements remain within 0.016 ms of the minimum for every GTR variant, indicating that the compiled forward path is stable rather than benefiting from an isolated timing outlier.

Small and medium models. GTR-S reaches 52.7 AP, only 0.2 AP below RF-DETR-S, while using 12.1M rather than 32M parameters and reducing minimum latency from 1.207 to 1.139 ms. It also obtains the strongest small-object result in its group, improving AP_S from the next-best 32.7 to 35.6. At the medium scale, GTR-M ranks first in overall AP, AP_{75} , AP_S , and AP_M . Its 55.6 AP exceeds RF-DETR-M by 0.9 AP with 11.3M fewer parameters, while its 1.350 ms minimum latency is only 0.024 ms above the fastest entry in the group.

Large and extra-large models. GTR-L provides the clearest joint improvement: it ranks first in minimum latency and in five of the six reported AP metrics, reaching 57.7 AP at 1.738 ms. Compared with D-FINE-L, it improves AP by 0.6 while running $1.30\times$ faster under our measurement protocol; compared with ECDet-L, the gain is 0.7 AP while running $1.43\times$ faster. At the extra-large scale, GTR-X prioritizes efficiency: its 1.931 ms minimum latency is the best in the group, while retaining 58.1 AP with 46.4M parameters. Relative to the higher-accuracy D-FINE-X, GTR-X is $1.72\times$ faster and uses 15.6M fewer parameters at a cost of 1.2 AP; relative to RF-DETR-X, it is $1.54\times$ faster with a 0.5 AP gap. These results make GTR-L the strongest joint accuracy-latency operating point in the family, while GTR-X offers a more latency-oriented option at higher capacity.

Table 1: Comparison with real-time object detectors on COCO val2017, grouped by model scale. Latency is measured on an RTX 4090 using PyTorch, FP16, and batch size 1; detailed evaluation settings are provided in Appendix B. The epoch column indicates the COCO training schedule, where * denotes additional epochs without strong augmentation. Within each scale, the top three results for minimum latency and each AP metric are highlighted in dark, medium, and light blue, respectively.

Model	#Epochs	#Params (M)	GFLOPs	Latency (ms) ↓			COCO AP ↑					
				Min.	Mean	Median	AP ^{val}	AP ^{val} ₅₀	AP ^{val} ₇₅	AP ^{val} _S	AP ^{val} _M	AP ^{val} _L
YOLOv9-S	500	7	26	1.651	2.326	2.294	46.8	61.8	48.6	25.7	49.9	61.0
YOLOv10-S	500	7	22	0.870	0.934	0.933	46.3	63.0	50.4	26.8	51.0	63.8
YOLO11-S	500	9	22	1.078	1.523	1.490	46.6	63.4	50.3	28.7	51.3	64.1
YOLOv12-S-Turbo	600	9	19	1.339	1.802	1.772	47.6	64.5	51.5	28.3	52.7	65.9
RT-DETRv2-S	120	20	60	1.184	1.279	1.291	48.1	65.1	52.1	30.2	51.5	63.9
DEIM-S	132*	10	25	1.090	1.171	1.180	49.0	65.9	53.1	30.4	52.6	65.7
DEIMv2-S	132*	10	26	1.550	1.676	1.687	50.9	68.4	55.1	31.3	55.3	70.2
RT-DETRv4-S	132*	10	25	1.079	1.166	1.174	49.7	66.8	54.1	30.2	53.6	66.9
LW-DETR-S	60	15	17	1.035	1.115	1.123	48.0	66.9	51.7	26.8	52.5	65.5
D-FINE-S	–	10	25	1.103	1.198	1.194	50.7	67.6	55.1	32.7	54.6	66.5
RF-DETR-S	–	32	60	1.207	1.285	1.299	52.9	71.9	57.0	32.0	58.3	73.0
YOLO26-S	70	10	21	0.922	0.977	0.962	47.8	64.6	52.1	29.1	52.5	64.3
ECDet-S	74	10	26	1.511	1.641	1.652	51.7	69.4	55.8	32.3	56.4	70.5
GTR-S (ours)	30	12.1	33.87	1.139	1.149	1.149	52.7	70.5	56.9	35.6	57.2	69.9
YOLOv9-M	500	20	76	2.023	2.713	2.675	51.4	67.2	54.6	32.0	55.7	66.4
YOLOv10-M	500	15	59	1.415	1.437	1.426	51.1	68.1	55.8	33.8	56.5	67.0
YOLO11-M	500	20	68	1.643	2.086	2.061	51.2	67.9	55.3	33.0	56.7	67.5
YOLOv12-M-Turbo	600	20	60	1.917	2.368	2.342	52.5	69.9	57.1	35.2	57.8	69.7
RT-DETRv2-M	120	31	92	1.593	1.727	1.741	49.9	67.5	54.1	32.0	53.2	66.5
DEIM-M	102*	19	57	1.612	1.732	1.744	52.7	70.0	57.3	35.3	56.7	69.5
DEIMv2-M	102*	18	52	2.193	2.390	2.398	53.0	70.2	57.6	34.2	57.4	71.5
RT-DETRv4-M	102*	19	57	1.598	1.731	1.742	53.5	71.1	58.1	34.9	57.7	72.1
LW-DETR-M	60	28	43	1.342	1.433	1.449	52.6	69.9	56.7	32.6	57.7	70.7
D-FINE-M	–	19	57	1.603	1.732	1.740	55.1	72.6	59.7	37.9	59.4	71.7
RF-DETR-M	–	34	79	1.326	1.428	1.431	54.7	73.5	59.2	36.1	59.7	73.8
YOLO26-M	80	20	68	1.459	1.481	1.473	52.5	69.8	57.2	36.2	56.9	68.5
ECDet-M	62	18	53	1.901	2.080	2.095	54.3	72.2	58.7	35.9	59.1	72.7
GTR-M (ours)	30	22.7	62.68	1.350	1.357	1.358	55.6	73.4	60.2	38.2	60.4	72.8
YOLOv9-C	500	25	102	2.049	2.716	2.680	53.0	70.2	57.8	36.2	58.5	69.3
YOLOv10-L	500	24	120	2.042	2.059	2.056	53.2	70.1	58.1	35.8	58.5	69.4
YOLO11-L	500	25	87	2.195	2.641	2.617	53.4	70.1	58.2	35.6	59.1	69.2
YOLOv12-L-Turbo	600	27	82	2.879	3.328	3.305	53.8	71.0	58.6	36.9	59.4	71.0
RT-DETRv2-L	72	42	136	2.127	2.317	2.332	53.4	71.6	57.4	36.1	57.9	70.8
DEIM-L	58*	31	91	2.251	2.446	2.460	54.7	72.4	59.4	36.9	59.6	71.8
DEIMv2-L	68*	32	97	2.560	2.759	2.776	56.0	73.5	61.1	37.6	60.9	74.9
RT-DETRv4-L	58*	31	91	2.264	2.450	2.464	55.4	73.0	60.3	37.1	60.1	72.9
LW-DETR-L	60	47	72	2.010	2.152	2.174	56.1	74.6	60.9	37.2	60.4	73.0
D-FINE-L	–	31	91	2.253	2.452	2.462	57.1	74.7	62.0	40.0	61.5	74.2
RF-DETR-L	–	34	126	1.805	1.952	1.968	56.5	75.1	61.3	39.0	61.0	73.9
YOLO26-L	60	25	86	1.953	1.969	1.967	54.3	71.5	59.4	37.8	58.6	70.3
ECDet-L	50	31	101	2.493	2.713	2.730	57.0	75.1	61.7	38.7	62.5	75.0
GTR-L (ours)	30	37.2	106.05	1.738	1.748	1.749	57.7	75.6	62.8	40.2	62.6	74.6
YOLOv9-E	500	57	189	3.960	4.633	4.598	55.6	72.8	60.6	40.2	61.0	71.4
YOLOv10-X	500	30	160	2.462	2.484	2.482	54.4	71.3	59.3	37.0	59.8	70.9
YOLO11-X	500	57	195	3.331	3.771	3.747	54.7	71.6	59.5	37.7	59.7	70.2
YOLOv12-X-Turbo	600	59	185	4.350	4.791	4.769	55.4	72.5	60.3	38.9	60.8	70.9
RT-DETRv2-X	72	76	259	3.209	3.260	3.259	54.3	72.8	58.8	35.8	58.8	72.1
DEIM-X	58*	62	202	3.336	3.388	3.384	56.5	74.0	61.5	38.8	61.4	74.2
DEIMv2-X	58*	50	152	3.218	3.288	3.276	57.8	75.3	63.2	39.1	62.9	75.9
RT-DETRv4-X	58*	62	202	3.332	3.367	3.363	57.0	74.6	62.1	39.5	61.9	74.8
LW-DETR-X	60	118	174	3.643	3.689	3.686	58.3	76.9	63.3	40.9	63.3	74.8
D-FINE-X	–	62	202	3.331	3.397	3.385	59.3	76.8	64.6	42.3	64.2	76.4
RF-DETR-X	–	126	300	2.971	3.227	3.243	58.6	77.4	63.8	40.3	63.9	76.2
YOLO26-X	40	55	194	3.069	3.088	3.087	56.9	74.1	62.1	41.3	61.2	72.7
ECDet-X	50	49	151	2.950	3.025	3.021	57.9	76.0	62.9	38.7	63.4	76.1
GTR-X (ours)	30	46.4	129.97	1.931	1.946	1.947	58.1	76.0	63.2	40.9	63.2	75.6

3.3 Transfer to Instance Segmentation

Accuracy–efficiency trade-off. The distilled GTR representation transfers from detection to instance segmentation without supervision beyond COCO. As shown in Table 2, GTR-S/M/L/X reach 44.4/47.2/49.0/49.5 mask AP with minimum latencies of 1.368/1.673/2.057/2.253 ms, respectively. All four variants rank first in overall mask AP within their scale groups.

Table 2: Comparison with real-time instance segmentation models on COCO val2017, grouped by model scale. Latency is measured on an RTX 4090 using PyTorch, FP16, and batch size 1. The Extra Sup. column indicates supervision beyond COCO, where O365+SAM2 denotes Objects365 detector pre-training with SAM2 pseudo masks before COCO fine-tuning. Within each scale, the top three results for minimum latency and each mask AP metric are highlighted in dark, medium, and light blue, respectively.

Model	Extra Sup.	#Params (M)	GFLOPs	Latency (ms) ↓			Mask AP ↑					
				Min.	Mean	Median	AP ^{val}	AP ^{val} ₅₀	AP ^{val} ₇₅	AP ^{val} _S	AP ^{val} _M	AP ^{val} _L
YOLOv8-Seg-S	–	11.8	42.6	1.602	1.703	1.669	36.8	58.1	38.9	17.3	41.3	53.7
YOLO11-Seg-S	–	10.1	35.5	1.257	1.730	1.701	37.8	59.9	40.4	19.8	42.7	55.2
YOLO26-Seg-S	O365+SAM2	10.4	34.2	1.151	1.194	1.163	40.0	61.5	43.0	21.0	44.5	57.3
RF-DETR-Seg-S	O365+SAM2	33.7	70.6	1.339	1.427	1.445	43.1	66.2	45.9	21.9	48.5	64.1
ECInsSeg-S	–	10.3	33.1	1.626	1.739	1.755	43.0	65.7	46.0	20.8	46.3	65.9
GTR-S (ours)	–	12.6	46.6	1.368	1.377	1.377	44.4	66.8	47.6	23.1	47.7	66.3
YOLOv8-Seg-M	–	27.3	110.2	2.398	2.491	2.462	40.8	63.6	43.8	22.0	46.0	58.2
YOLO11-Seg-M	–	22.4	113.2	2.079	2.546	2.519	41.5	65.1	44.5	23.0	47.2	59.1
YOLO26-Seg-M	O365+SAM2	23.6	121.5	1.992	2.006	2.004	44.1	66.8	47.7	25.6	48.9	60.2
RF-DETR-Seg-M	O365+SAM2	35.7	102.0	1.548	1.647	1.672	45.3	68.4	48.8	25.5	50.4	65.3
ECInsSeg-M	–	20.1	64.2	2.077	2.252	2.252	45.2	68.2	48.3	22.9	49.0	68.1
GTR-M (ours)	–	23.7	83.9	1.673	1.696	1.696	47.2	70.5	50.9	26.0	50.7	68.4
YOLOv8-Seg-L	–	46.0	220.5	2.866	3.328	3.301	42.6	66.2	45.5	23.9	47.9	59.5
YOLO11-Seg-L	–	27.6	132.2	2.696	3.152	3.125	42.9	67.0	46.2	24.6	48.7	60.6
YOLO26-Seg-L	O365+SAM2	28.0	139.8	2.574	2.598	2.597	45.5	68.7	49.2	27.1	50.4	62.8
RF-DETR-Seg-L	O365+SAM2	36.2	151.1	1.907	2.033	2.055	47.1	70.5	50.9	28.4	52.1	65.6
ECInsSeg-L	–	33.6	110.8	2.661	2.862	2.869	47.1	70.9	50.5	24.8	51.1	69.6
GTR-L (ours)	–	38.2	127.3	2.057	2.092	2.090	49.0	72.7	53.2	27.8	52.9	70.4
YOLOv8-Seg-X	–	71.8	344.1	3.688	4.155	4.122	43.4	67.1	46.5	25.6	48.9	60.4
YOLO11-Seg-X	–	62.1	296.4	4.161	4.607	4.584	43.8	68.5	47.1	25.7	49.7	61.4
YOLO26-Seg-X	O365+SAM2	62.8	313.5	4.096	4.143	4.139	47.0	70.8	51.1	29.7	51.8	63.1
RF-DETR-Seg-X	O365+SAM2	38.1	260.0	3.064	3.299	3.311	48.8	72.2	53.1	30.6	53.3	65.9
ECInsSeg-X	–	49.9	168.1	3.125	3.147	3.140	48.4	72.2	52.0	26.3	52.7	71.1
GTR-X (ours)	–	47.4	151.2	2.253	2.277	2.277	49.5	73.2	53.9	29.0	53.6	71.3

Comparison across scales. RF-DETR-Seg uses Objects365 pre-training and SAM2 pseudo masks. Without this extra supervision, GTR surpasses RF-DETR-Seg by 1.3/1.9/1.9/0.7 mask AP at the S/M/L/X scales. GTR-S achieves this gain with 12.6M parameters, compared with 33.7M for RF-DETR-Seg-S. GTR-M and GTR-L rank second in minimum latency and lead all six mask AP metrics, with one exception: GTR-L is second in AP_S. At the extra-large scale, GTR-X is both the fastest and most accurate model in the group; its 2.253 ms minimum latency is 1.36× faster than RF-DETR-Seg-X (3.064 ms).

3.4 Ablation Study

Protocol. All ablations use GTR-S. For each variant, we distill the backbone with the indicated configuration, omit Objects365 detector pre-training, and train the downstream detector directly on COCO. All other settings are kept fixed, and the tables report box AP on val2017.

Distillation mask ratio. The mask ratio controls how much visual context is available to the student during distillation. Table 3 shows that a ratio of 0.5 yields the highest overall AP of 51.9. It also achieves the best AP₅₀, AP₇₅, and AP_L. Performance remains stable between 0.3 and 0.7, ranging from 51.6 to 51.9 AP. AP drops to 51.5 and 51.3 at ratios of 0.2 and 0.8, respectively. We therefore use 0.5 as the default ratio.

Distillation losses. Table 4 isolates the contributions of masked token prediction, VRA, and intermediate Gram supervision. Masked token prediction alone yields 49.0 AP. Adding VRA or Gram supervision raises AP to 51.5 and 51.7, corresponding to gains of 2.5 and 2.7 points, respectively. Combining all three losses yields 51.9 AP, a 2.9-point gain over masked token prediction alone, together with the best AP₅₀, AP₇₅, AP_M, and AP_L. These results show that visible-token alignment and relational supervision provide complementary constraints for distilling the self-attention teacher into the GLA student.

Table 3: Effect of the distillation mask ratio. Each GTR-S detector is trained directly on COCO without Objects365 detector pre-training; the selected default is shaded.

Mask ratio	AP^{val}	AP_{50}^{val}	AP_{75}^{val}	AP_S^{val}	AP_M^{val}	AP_L^{val}
0.2	51.5	69.4	55.6	33.4	55.5	69.3
0.3	51.7	69.3	55.9	34.8	56.1	69.0
0.4	51.7	69.5	55.9	33.8	55.8	69.3
0.5	51.9	69.7	56.1	33.5	56.1	70.4
0.6	51.6	69.1	55.8	33.6	55.8	70.0
0.7	51.6	69.2	55.9	32.9	55.4	69.8
0.8	51.3	69.0	55.5	33.1	55.6	69.7

Table 4: Effect of the distillation losses. A filled circle indicates that the loss is enabled; all detectors are trained directly on COCO without Objects365 detector pre-training. The default configuration is shaded.

$\mathcal{L}_{mask}$	$\mathcal{L}_{vra}$	$\mathcal{L}_{gram}$	AP^{val}	AP_{50}^{val}	AP_{75}^{val}	AP_S^{val}	AP_M^{val}	AP_L^{val}
•	–	–	49.0	66.2	52.9	31.2	52.5	66.7
•	•	–	51.5	69.2	55.8	32.9	55.7	69.5
•	–	•	51.7	69.3	55.6	34.0	55.7	70.2
•	•	•	51.9	69.7	56.1	33.5	56.1	70.4

References

- Aviv Bick, Kevin Y. Li, Eric P. Xing, J. Zico Kolter, and Albert Gu. Transformers to ssms: Distilling quadratic knowledge to subquadratic models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-0999.
- Aviv Bick, Tobias Katsch, Nimit Sharad Sohoni, Arjun D. Desai, and Albert Gu. Llama: Scaling distilled recurrent models for efficient language processing. In *ICLR 2025 Poster*, 2025. Published: 6 March 2025; last modified: 15 April 2025.
- Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision*, pp. 213–229. Springer, 2020.
- Qiang Chen, Xiaokang Chen, Jian Wang, Shan Zhang, Kun Yao, Haocheng Feng, Junyu Han, Errui Ding, Gang Zeng, and Jingdong Wang. Group detr: Fast detr training with group-wise one-to-many assignment. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2023.
- Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision*, pp. 740–755, 2014.
- Longfei Liu, Yongjie Hou, Yang Li, Qirui Wang, Youyang Sha, Yongjun Yu, Yinshi Wang, Peizhe Ru, Xuanlong Yu, and Xi Shen. Edgecrafter: Compact vits for edge dense prediction via task-specialized distillation. *arXiv*, 2026.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Yansong Peng, Hebei Li, Peixi Wu, Yueyi Zhang, Xiaoyan Sun, and Feng Wu. D-fine: Redefine regression task of detr as fine-grained distribution refinement. In *International Conference on Learning Representations*, 2025. Spotlight.
- Zhiliang Peng, Li Dong, Hangbo Bao, Furu Wei, and Qixiang Ye. A unified view of masked image modeling. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=wmGIMhaBe0>.

- Shuai Shao, Zeming Li, Tianyuan Zhang, Chao Peng, Gang Yu, Xiangyu Zhang, Jing Li, and Jian Sun. Objects365: A large-scale, high-quality dataset for object detection. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 8430–8439, 2019.
- Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. DINOv3, 2025. URL <https://arxiv.org/abs/2508.10104>.
- Junxiong Wang, Daniele Paliotta, Avner May, Alexander M. Rush, and Tri Dao. The mamba in the llama: Distilling and accelerating hybrid models. In Advances in Neural Information Processing Systems, volume 37, 2024. doi: 10.52202/079017-1996.
- Guoyizhe Wei and Rama Chellappa. Vit-linearizer: Distilling quadratic knowledge into linear-time vision models. In Proceedings of the IEEE/CVF International Conference on Computer Vision, 2025.
- Songlin Yang and Yu Zhang. Fla: A triton-based library for hardware-efficient implementations of linear attention mechanism, January 2024. URL <https://github.com/fla-org/flash-linear-attention>.
- Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In Proceedings of ICML, 2024.

Table 5: Architecture configurations of the four GTR variants.

Setting	GTR-S	GTR-M	GTR-L	GTR-X
Embedding dimension	192	256	384	384
GLA heads	3	4	6	6
Backbone MLP ratio	4	4	4	6
Encoder width	192	256	256	256
Decoder FFN dimension	512	1024	1024	2048
Decoder layers	4	4	4	4
Object queries	300	300	300	300
Training query groups	3	3	3	3

Table 6: Objects365 detector pre-training hyperparameters.

Setting	GTR-S	GTR-M	GTR-L	GTR-X
Epochs	36	36	36	36
Global batch size	128	128	128	128
Base LR	2×10^{-3}	2×10^{-3}	2×10^{-3}	2×10^{-3}
Backbone LR	1×10^{-4}	1×10^{-4}	2×10^{-5}	1×10^{-5}
Weight decay	1×10^{-4}	1×10^{-4}	1.25×10^{-4}	1.25×10^{-4}
AdamW β	0.9/0.999	0.9/0.999	0.9/0.999	0.9/0.999
LR schedule	flat-cosine	flat-cosine	flat-cosine	flat-cosine
Flat phase ends at epoch	15	15	15	15
Minimum/base LR ratio	0.1	0.1	0.1	0.1
Warm-up iterations	2000	2000	2000	2000

A Detailed Experimental Configurations

A.1 Model Scaling

All GTR variants share a 12-layer, stride-16 GLA backbone and extract multi-scale features from intermediate layers. Their scale-specific architecture configurations are summarized in Table 5.

A.2 Objects365 Pre-training

Each detector is initialized with its corresponding masked-distilled GLA backbone and subsequently pre-trained on Objects365. The complete training configuration is provided in Table 6.

A.3 COCO Fine-tuning

For COCO fine-tuning, we transfer only the backbone weights from the best Objects365 checkpoint, while initializing the multi-scale encoder and detection decoder from scratch. All variants are fine-tuned on train2017 and evaluated on val2017; detailed settings are summarized in Table 7.

B Latency Measurement Protocol

B.1 Hardware, Software, and Deployment

All detector latency experiments are conducted on a single NVIDIA GeForce RTX 4090 GPU with Python 3.11, PyTorch 2.11.0+cu128, torchvision 0.26.0+cu128, and CUDA 12.8. The batch size is fixed to one. RF-DETR-S/M/L/X use their native resolutions of 512/576/704/700, respectively; every other model uses a 640×640 input.

Table 7: COCO fine-tuning hyperparameters. Bias and normalization parameters use zero weight decay in both training stages.

Setting	GTR-S	GTR-M	GTR-L	GTR-X
Epochs	30	30	30	30
Global batch size	32	32	32	32
Base LR	5×10^{-4}	5×10^{-4}	5×10^{-4}	5×10^{-4}
Backbone LR	1.5×10^{-5}	1.5×10^{-5}	2.5×10^{-6}	2×10^{-6}
Weight decay	1×10^{-4}	1×10^{-4}	1.25×10^{-4}	1.25×10^{-4}
LR schedule	flat-cosine	flat-cosine	flat-cosine	flat-cosine
Flat phase ends at epoch	6	6	6	6
Minimum/base LR ratio	0.5	0.5	0.5	0.1
Warm-up iterations	2000	2000	2000	2000

All models run in FP16. We first invoke the official `deploy()`, `export()`, or `fuse()` path, when available, to apply deployment conversion and Conv-BN fusion. For implementations without an explicit deployment interface, TorchInductor freezing is enabled. Every model is compiled with

```
torch.compile(mode="max-autotune", fullgraph=True)
```

and executed through a CUDA Graph. The `fullgraph=True` constraint turns any data-dependent graph break or device synchronization into a compilation error rather than silently falling back to eager execution. PyTorch profiler traces verify that, in steady state, one forward call produces exactly one CUDA-Graph launch, zero GPU→CPU scalar synchronizations, and zero device-to-host copies.

The official RF-DETR and LW-DETR implementations use GPU-resident shape scalars in split/view operations and perform runtime assertions on tensors, which prevents full-graph compilation. We replace those values with compile-time shape constants before benchmarking. The transformation is numerically equivalent: the outputs before and after the change are bitwise identical. Compilation, autotuning, graph capture, and all warm-up time are excluded from the reported latency.

B.2 Timed Scope and Post-processing

For DETR-style models, we time only the model forward pass and exclude output decoding and other post-processing. For YOLOv9, YOLO11, and YOLOv12, which require non-maximum suppression, the reported value includes both the forward pass and CUDA NMS implemented by `torchvision.ops.nms`. The NMS parameters are `conf=0.001`, `iou=0.7`, `multi_label=True`, `agnostic=False`, and `max_det=300`. Because NMS cost depends on the number of candidate boxes, these models are measured image by image over all 5000 images in COCO val2017. YOLOv10 and YOLO26 perform top- k selection inside the network and have no separate NMS stage. Models without NMS are measured on a fixed set of 1000 COCO images.

B.3 Warm-up and Statistics

YOLO models receive 200 forward warm-up iterations before measurement. YOLO11 and YOLO12 additionally run 30 real-prediction NMS warm-up iterations. YOLOv9 does not receive a separate NMS warm-up; averaged over 5000 images, its one-time cold-start contribution is negligible. All other models receive 300 warm-up iterations.

Timing is performed on the default CUDA stream while the GPU is otherwise idle. Each model is measured once in an independent process, and Table 1 reports the minimum, arithmetic mean, and median per-image latency. The raw records additionally retain the standard deviation and the full CUDA-event sequence. Independent spot checks repeated on the same day vary by approximately $\pm 2\%$ across processes.

C Custom CUDA Operator Benchmark

The custom chunk-based GLA operator and the FLA baseline Yang & Zhang (2024) are benchmarked on a single NVIDIA GeForce RTX 4090 with 24GB memory and compute capability sm_89. The software environment is Linux x86-64, CUDA Toolkit 12.8 (nvcc 12.8.93), Python 3.13, PyTorch 2.11.0 with cu128, Triton 3.6.0, and flash-linear-attention 0.5.0. The Triton version is fixed by the PyTorch 2.11.0 dependency specification. Our operator is compiled in place as a PyTorch CUDA extension with -O3, fast math, and targets sm_80, sm_86, and sm_89.

The timing setup matches the deployment path. For both implementations, one CUDA Graph captures 12 consecutive operator calls. We perform 20 replay warm-ups and then time 200 replays with CUDA Events, reporting the mean latency per operator call in microseconds. Grouping 12 calls per graph amortizes graph-replay and launch overhead and exposes the steady-state operator cost within a captured inference pipeline.

The input shape is fixed to

$$B = 1, \quad H = 6, \quad K = 32, \quad V = 64, \quad (18)$$

with chunk size 64 and FP16 inputs. We sweep the sequence length L over $\{256, 512, 1024, 2048, 2917, 4096, 8192, 16384\}$. As shown in Figure 4, the custom implementation is faster at every tested length and reaches a $6.3\times$ speedup at $L = 16,384$.